

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 3

The Command Line Shell, Pipes and Text Filters

Ciur Alexandrina FAF-241

Checked by:
Reitamn Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 ALGORITHM ANALYSIS **1**
- 1.1 Objective . . . 1
- 1.2 Tasks . . . 1
- 1.3 Theoretical Notes . . . 1
- 1.4 Introduction . . . 2
- 1.5 Comparison Metric . . . 2
- 1.6 Input Format . . . 2

2 IMPLEMENTATION **3**
- 2.1 Command-Line Solutions . . . 3
  - 2.1.1 Question 1: Number of Failed Requests . . . 3
  - 2.1.2 Question 2: Different Requested Pages . . . 3
  - 2.1.3 Question 3: Number of Requests for Each Page . . . 3
  - 2.1.4 Question 4: Users with the Most Failed Requests . . . 3
  - 2.1.5 Question 5: Requests Made by user3 . . . 3
  - 2.1.6 Question 6: Last Three Failed Requests . . . 3
  - 2.1.7 Question 7: Login Shells from /etc/passwd . . . 4
  - 2.1.8 Question 8: Comparing the Number of Lines . . . 4
- 2.2 Bash Report Script . . . 5
- 2.3 Explanation of the Script . . . 5
- 2.4 Running the Script . . . 6

3 CONCLUSIONS **7**

1 ALGORITHM ANALYSIS

1.1 Objective

The objective of this laboratory work is to practice using the Linux command line shell, pipes, redirection, and standard text-processing utilities.

The laboratory also demonstrates how several simple commands can be combined to process log files and system information efficiently without writing a large program.

1.2 Tasks

1. Study basic shell commands and identify which commands are shell built-ins and which are external programs.
2. Practice using the commands `grep`, `cut`, `sort`, `uniq`, `wc`, `tail`, and `ls`.
3. Use pipelines to process the file `access.log` and answer questions about requests, users, pages, and failed operations.
4. Create a Bash script that receives a log file as an argument and prints the total number of requests and the number of failed requests for every user.

1.3 Theoretical Notes

A shell is a command interpreter that allows a user to communicate with the operating system. It reads commands, executes programs, and displays their output.

A pipe is represented by the character `|`. It connects the standard output of one command to the standard input of another command.

For example:

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c
```

The commands in this pipeline perform the following operations:

- `grep FAIL access.log` selects lines containing `FAIL`;
- `cut -d' ' -f3` extracts the third field;
- `sort` sorts the extracted values;
- `uniq -c` counts equal consecutive values.

The most important commands used in this laboratory are:

- `grep` – searches for lines matching a pattern;
- `cut` – extracts fields or characters from each line;
- `sort` – sorts lines of text;
- `uniq` – removes or counts repeated lines;
- `wc` – counts lines, words, or characters;
- `tail` – displays the last lines of a file;
- `ls` – lists directory contents.

1.4 Introduction

The input file used in this laboratory is an access log named `access.log`. Each line contains information about a request, including the time, user, requested page, and request status.

The log file is processed using pipelines. Instead of manually inspecting each line, filters are combined to count requests, identify pages, determine which users had failed requests, and display the latest failed operations.

The laboratory also uses the file `/etc/passwd` to demonstrate how system account information can be processed using text filters.

1.5 Comparison Metric

The correctness of the solution is evaluated by comparing the output of the commands and the Bash script with the contents of the input file.

The following results are verified:

- total number of requests;
- number of failed requests;
- number of different requested pages;
- number of requests for each page;
- number of failed requests for each user;
- last three failed requests;
- number of accounts using each login shell;
- difference between `ls /etc | wc -l` and `ls -l /etc | wc -l`.

1.6 Input Format

The program receives the log file as an input argument:

```
./report.sh access.log
```

The expected format of the log file is a text file with one request per line. The fields are separated by spaces.

The script expects the following users to occur in the log file:

```
user0 user1 user2 user3 user4 user5 user6
```

The script verifies that the file exists before processing it.

2 IMPLEMENTATION

2.1 Command-Line Solutions

The following commands answer the questions based on the contents of `access.log`.

2.1.1 Question 1: Number of Failed Requests

```
grep -c FAIL access.log
```

The option `-c` counts the number of matching lines. Therefore, the command counts all requests containing the word `FAIL`.

2.1.2 Question 2: Different Requested Pages

```
cut -d' ' -f5 access.log | sort -u
```

The fifth field contains the requested page. The option `-u` removes duplicate values after sorting.

2.1.3 Question 3: Number of Requests for Each Page

```
cut -d' ' -f5 access.log | sort | uniq -c
```

The pages are first sorted because `uniq` only counts adjacent equal lines. The option `-c` displays the number of occurrences.

2.1.4 Question 4: Users with the Most Failed Requests

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
```

This pipeline selects failed requests, extracts the user field, counts the requests for every user, and sorts the result in descending numerical order.

2.1.5 Question 5: Requests Made by user3

```
echo "Total requests:"
grep -c 'user3' access.log

echo "Failed requests:"
grep 'user3' access.log | grep -c FAIL
```

The first command counts all requests belonging to `user3`. The second command counts only those requests that also contain `FAIL`.

2.1.6 Question 6: Last Three Failed Requests

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

The command selects failed requests, keeps the final three lines, and displays only the second and third fields, representing the time and user.

2.1.7 Question 7: Login Shells from /etc/passwd

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

The fields in `/etc/passwd` are separated by colons. The seventh field contains the login shell. The command counts the number of accounts using each shell.

2.1.8 Question 8: Comparing the Number of Lines

```
ls /etc | wc -l
ls -l /etc | wc -l
```

The first command counts directory entries displayed by the normal `ls` command. The second command counts lines displayed by the long-format command.

The results can differ because `ls` may format output in columns, while `ls -l` normally displays one entry per line. In addition, the long-format listing may contain a total line.

2.2 Bash Report Script

The following script receives the log file as an argument and displays the total number of requests, the number of failed requests, and the number of failed requests for each user.

```
#!/bin/bash

# Usage:
# ./report.sh access.log

log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")

echo "$total requests, $fails failed"

for user in user0 user1 user2 user3 user4 user5 user6
do
    count=$(grep "$user" "$log" | grep -c FAIL)
    echo "$user: $count failures"
done
```

2.3 Explanation of the Script

The first line identifies the Bash interpreter:

```
#!/bin/bash
```

The input file is stored in the variable `log`:

```
log=$1
```

Here, `$1` represents the first command-line argument.

The following condition verifies that the specified file exists:

```
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
```

The total number of lines is calculated with `wc -l`. Input redirection is used so that the file name is not included in the output:

```
total=$(wc -l < "$log")
```

The number of failed requests is calculated using `grep -c`:

```
fails=$(grep -c FAIL "$log")
```

Finally, a `for` loop processes all users from `user0` to `user6`. For each user, the script counts lines containing both the user name and the word `FAIL`.

2.4 Running the Script

The script can be executed with the following commands:

```
chmod +x report.sh
./report.sh access.log
```

The expected output has the following form:

```
<number> requests, <number> failed
user0: <number> failures
user1: <number> failures
user2: <number> failures
user3: <number> failures
user4: <number> failures
user5: <number> failures
user6: <number> failures
```

3 CONCLUSIONS

During this laboratory work, Bash commands, pipes, and text-processing utilities were used to analyze an access log and system account information.

The commands `grep`, `cut`, `sort`, `uniq`, `tail`, and `wc` were combined into pipelines. This made it possible to answer several questions about the log file using short and readable commands.

A Bash script named `report.sh` was also implemented. The script accepts a file name as an argument, checks whether the file exists, counts the total and failed requests, and displays the number of failed requests for each user.

The laboratory demonstrated that shell pipelines are useful for processing structured text efficiently. It also showed the importance of sorting data before using `uniq`, validating input files, and understanding the difference between normal and long-format directory listings.